

Lambda Functions: Anonymous Functions in Python.

Python Built-In Functions: map, filter, and reduce

Lambda functions, also known as anonymous functions, are small, unnamed functions defined using the lambda keyword. They are typically used within other functions or as arguments to higher-order functions.

The map(), filter(), and reduce() functions are powerful tools in Python for manipulating iterables. They allow you to transform, filter, and combine elements in a concise and expressive way. Let's explore each of these functions and their usage.



by Prof. Muhammad Iqbal Bhat
GDC Beerwah

```
unitte
Goog

915
916
917
918
919
920
921
922
923
924
925
926
927
928
929
930
931
932
933
934
935
936
937
938
939
940
941
942
943
944
945
946
947
948

return

source = request.get('sour
payload_json = request.get

student = models.Student.g
if student:
    payload = transforms.l
    self.addDeviceData(pay
self.process_event(user, s

# models.EventEntity.addDe
if student:
    models.EventEntity.rec
else:
    models.EventEntity.rec
COURSE_EVENTS_RECORDED.inc

self.process_event(user, s

def process_event(self, user,
    """Processes an event after

student = models.Student.g
if not student:
    return

payload = transforms.loads

self.addDeviceData(payload

if 'location' not in paylo
return
```

What are Lambda Functions?

- **Lambda functions**, also known as **anonymous functions** or **lambda expressions**, are small, anonymous functions defined using the lambda keyword.
- They are commonly used for short, simple operations where you don't want to define a full function using def.
- Lambda functions are particularly useful when you need a quick, one-time function for tasks like sorting, filtering, or mapping elements of data structures.

Syntax:

```
lambda arguments: expression
```

- **lambda:** The keyword used to define a lambda function.
- **arguments:** The input parameters to the function, separated by commas.
- **expression:** The single expression that the function will evaluate and return.

Examples of Lambda Functions

1 Add two numbers:

```
add = lambda x, y: x + y  
result = add(3, 5)
```

2 Square a number:

```
add = lambda x, y: x + y  
result = add(3, 5)
```

3 Mapping to absolute values:

```
values = [-4, -2, 0, 2, 4]  
absolute_values = map(lambda x: abs(x), values)  
print(list(absolute_values))
```



Characteristics of Lambda Functions

Anonymous

No explicit name

Single-Expression

Contain only one expression

Implicit Return

Automatically return the evaluated expression

Benefits of Lambda Functions

Concise

Allow for defining short, inline functions

Readability

Enhance code readability by keeping related logic together

Flexibility

Can be passed as arguments to other functions



Applications of Lambda Functions

1

Higher-Order Functions:

Lambda functions are often used as arguments to higher-order functions.

2

Event Handling:

Lambda functions are commonly used in event handling to define callback functions for specific events.

3

Data Transformation:

Lambda functions can be employed for concise data transformation tasks.

4

Customizations:

Lambda functions provide flexibility in customizing behavior.

map() Function

The `map()` function is a higher-order function that applies a given function to each element of an iterable and returns a new iterable containing the transformed elements.

Syntax:

```
map(function, iterable)
```

For example:

```
numbers = [1, 2, 3, 4]
squared_numbers = map(lambda x: x * x, numbers)
print(list(squared_numbers))
```

Output:

```
[1, 4, 9, 16]
```

map() Function

1. Converting Data Types: Convert the elements of a list from strings to integers:

```
string_values = ["1", "2", "3", "4", "5"]
integer_values = map(lambda x: int(x), string_values)
print(list(integer_values))
```

Output:

```
[1, 2, 3, 4, 5]
```

2. Customizing Transformations: Perform a custom transformation on each element of a list::

```
data = ["apple", "orange", "banana"]
modified_data = map(lambda x: x.upper() + "!", data)
print(list(modified_data))
```

Output:

```
['APPLE!', 'ORANGE!', 'BANANA!']
```


Use Cases for map()

The `map()` function is versatile and can be used in various scenarios. Here are some common use cases:

- Applying a mathematical operation to a list of numbers
- Transforming strings into uppercase or lowercase
- Converting between different data types
- Mapping values to corresponding keys in dictionaries

By leveraging the power of `map()`, you can efficiently process your data and achieve the desired results.

filter() Function

The filter() function is another powerful tool in Python. It takes a function to evaluate and an iterable to filter. It applies the given function to each element in the iterable and returns a new iterable containing only the elements for which the function returns True.

Syntax:

```
filter(function, iterable)
```

For example:

```
numbers = [1, 2, 3, 4, 5]
even_numbers = filter(lambda x: x % 2 == 0, numbers)
print(list(even_numbers))
```

Output:

```
[2, 4]
```

filter() Function

1. Filtering Strings: Identify strings with a specific length:

```
words = ["hello", "world", "python", "programming"]  
long_words = filter(lambda x: len(x) >= 6, words)  
print(list(long_words))
```

Output:

```
['python', 'programming']
```

2. Customizing Selection Criteria: Apply a custom condition to filter elements:

```
temperatures = [25, 18, 32, 29, 22]  
comfortable_temperatures = filter(lambda x: 20 <= x <= 27, temperatures)  
print(list(comfortable_temperatures))
```

Output:

```
[25, 22]
```

Use Cases for filter()

The filter() function is valuable for filtering iterables based on certain conditions. Here are some practical use cases:

- Extracting even or odd numbers from a list
- Filtering out missing or invalid data
- Removing duplicates from a sequence
- Selecting rows from a dataset based on specific criteria

With filter(), you can easily extract the relevant elements from your data and focus on the information that matters.

reduce() Function

The `reduce()` function is used to progressively reduce an iterable to a single value by applying a function to two elements at a time. It starts with the first element as the initial value and passes the result of each application as the first argument in the next iteration.

Syntax:

```
reduce(function, iterable)
```

For example:

```
numbers = [1, 2, 3, 4, 5]  
sum_of_numbers = reduce(lambda x, y: x + y, numbers)  
print(sum_of_numbers)
```

Output:

```
15
```

reduce() Function

1. Product of Values: Compute the product of all elements in a list:

```
values = [2, 3, 4, 5]
product = reduce(lambda x, y: x * y, values)
print(product)
```

Output:

```
120
```

2. Customizing Reduction Operation: Apply a custom operation to accumulate values::

```
expenses = [100, 50, 80, 120]
average_expense = reduce(lambda x, y: (x + y) / 2, expenses)
print(average_expense)
```

Output:

```
85.0
```

Use Cases for reduce()

The `reduce()` function is powerful for aggregating data into a single value. Here are some examples of its application:

- Calculating the sum or product of a sequence of numbers
- Finding the maximum or minimum value in a list
- Performing financial calculations like compound interest
- Computing statistical metrics such as mean or standard deviation

By employing `reduce()`, you can perform complex computations on your data and derive valuable insights.

Relationship between Map, Filter, and Reduce

These three functions – `map()`, `filter()`, and `reduce()` – are powerful tools for manipulating iterables in Python. They can be combined to create more complex operations and are often used together in functional programming.

- `map()` is used to transform elements.
- `filter()` is used to select elements based on a condition.
- `reduce()` is used to combine elements to a single value.

Applications of Map, Filter, and Reduce

The `map()`, `filter()`, and `reduce()` functions find applications in various tasks like data transformation, filtering, and aggregation.

Data Transformation: Use `map()` to apply a function to each element and transform the data.

Filtering: Use `filter()` to select elements that satisfy a specific condition.

Aggregation: Use `reduce()` to combine the elements of an iterable into a single value.

By utilizing these functions, you can create efficient and concise code for complex operations on your data.

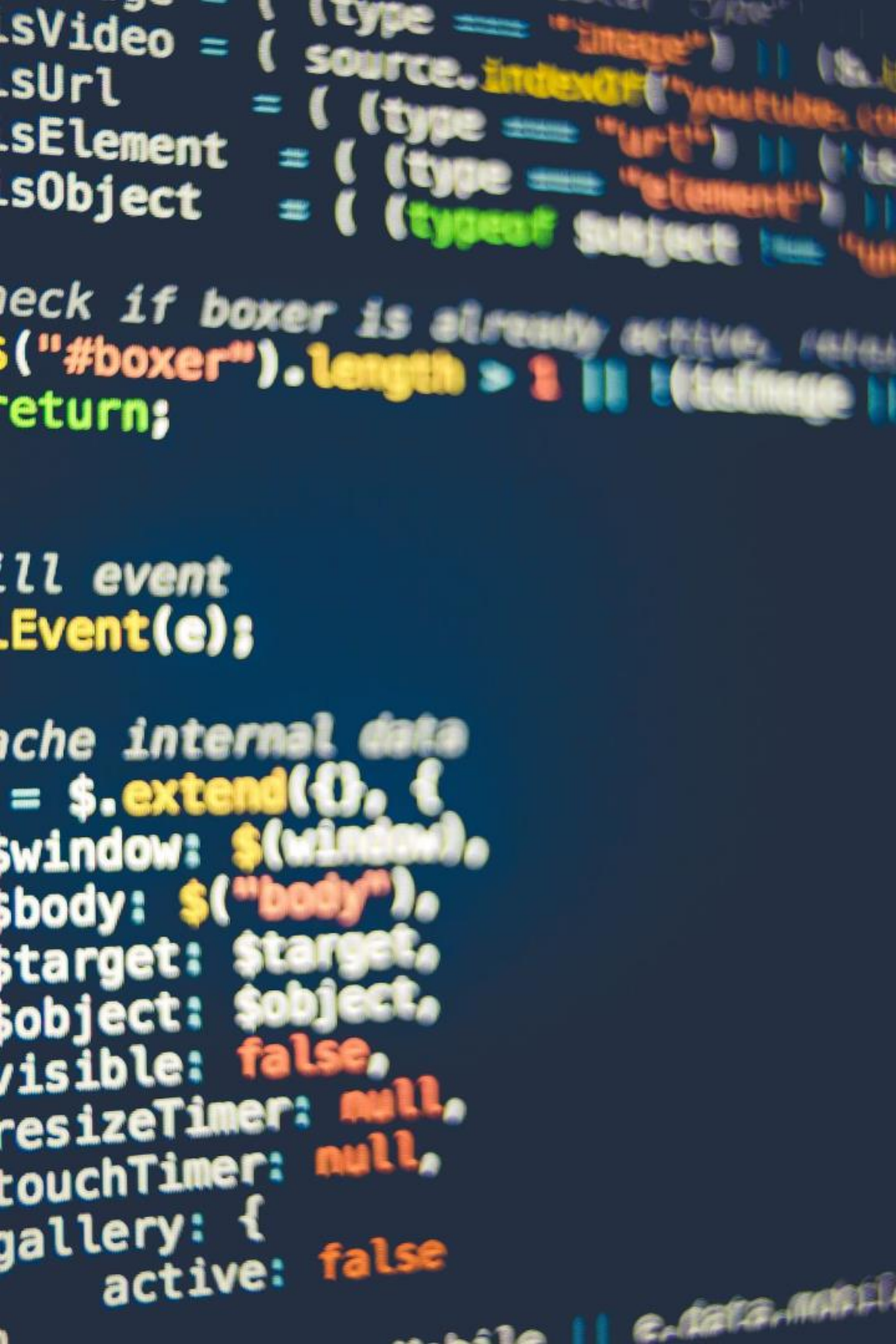
```
* @param context Context to get resources and device specific display metrics
* @return A float value to represent px equivalent to dp depending on device
density
*/
public static int convertDpToPx(int dp) {
    Resources resources = Resources.getSystem();
    DisplayMetrics metrics = resources.getDisplayMetrics();
    int px = dp * (metrics.densityDpi / DisplayMetrics.DENSITY_DEFAULT);
    return px;
}

/**
 * This method converts device specific pixels to density independent pixels.
 *
 * @param px A value in px (pixels) unit. Which we need to convert into dp
 * @param context Context to get resources and device specific display metrics
 * @return A float value to represent dp equivalent to px value
 */
```

The Power of Map, Filter, and Reduce

The `map()`, `filter()`, and `reduce()` functions are essential tools for functional programming in Python. They provide a concise and expressive way to manipulate iterables, making them valuable for various programming tasks.

By using these functions, you can enhance the readability and maintainability of your code while achieving efficient data processing.



The Future of Map, Filter, and Reduce

Map, filter, and reduce are foundational concepts in functional programming that have influenced modern programming languages. As Python continues to evolve, these functions remain relevant and provide efficient ways to process data.

By embracing the power of `map()`, `filter()`, and `reduce()`, you can write clear, concise, and performant code.